

Visão do Projeto

1. Introdução

Este documento descreve a visão geral para o projeto **track'd**, um sistema de rastreamento de produção por RFID. O objetivo é desenvolver uma solução integrada para monitorar em tempo real o fluxo de itens em linhas de produção, utilizando tags RFID, um backend em Python com banco de dados PostgreSQL e um frontend em Svelte para visualização. O projeto inicial focará em entregar as funcionalidades críticas de rastreamento, com uma visão de longo prazo para se tornar uma ferramenta analítica essencial para a otimização de processos industriais.

2. Problema

Atualmente, o acompanhamento de itens ao longo da linha de produção enfrenta desafios significativos que geram ineficiência e custos:

- **Falta de Visibilidade em Tempo Real:** A gestão da produção frequentemente não sabe a localização exata de um item ou lote, resultando em atrasos na busca por materiais.
- **Rastreamento Manual e Sujeito a Erros:** Apontamentos manuais de entrada e saída de itens em setores são lentos, consomem mão de obra e são uma fonte constante de erros de registro.
- **Dificuldade em Identificar Gargalos:** Sem dados precisos sobre o tempo de permanência de itens em cada etapa (câmara), é difícil identificar e corrigir gargalos que limitam a capacidade produtiva.
- **Ausência de Histórico para Análise:** A falta de um histórico de dados consolidado impede análises aprofundadas sobre a eficiência do fluxo de produção e a otimização de layouts e processos.

3. Público-Alvo e Stakeholders

- **Supervisores de Produção (Público Principal):** Necessitam de uma visão clara e instantânea do status da linha para gerenciar equipes, tomar decisões rápidas e garantir o cumprimento das metas.
- **Operadores de Linha:** Utilizarão o sistema para confirmar o movimento de peças e necessitam de uma ferramenta que seja transparente e não interfira em seu trabalho.

- **Gerentes de Planta/Logística:** Interessados nos dados agregados, como tempo médio de ciclo, produtividade por setor e relatórios de eficiência para planejamento estratégico.
- **Equipe de TI e Manutenção:** Responsáveis pela infraestrutura de rede, servidores, banco de dados e manutenção das antenas RFID.

4. Visão do Produto

Para os **supervisores de produção** que precisam de visibilidade e controle precisos sobre o fluxo de trabalho, o **track'd** é uma **plataforma de monitoramento industrial** que automatiza o rastreamento de itens através de leituras RFID. Diferente de **sistemas de controle manuais ou planilhas**, nosso produto oferece um **dashboard em tempo real com a localização exata, tempo de permanência e histórico completo de cada item**, permitindo a identificação imediata de gargalos e a otimização contínua do processo produtivo.

5. Escopo e Funcionalidades

5.1. Escopo Inicial (MVP - Produto Mínimo Viável) O foco é entregar um sistema robusto e funcional que resolva o problema central de rastreamento.

- **Backend de Processamento (Python):** Serviço que recebe os dados brutos das antenas, processa-os e armazena de forma estruturada no banco de dados.
 - **Banco de Dados (PostgreSQL):** Armazenamento seguro de todas as leituras, registrando ID da tag, ID da câmara, antena, horário de entrada/saída e supervisor responsável. (DADOS DO PRODUTO; E DADOS DO CLIENTE-PROPRIETÁRIO DO PRODUTO)
 - **Frontend de Monitoramento (Svelte):** Um dashboard web para visualização dos dados.
 - Tela de status atual mostrando quais tags estão em cada câmara.
 - Tela de consulta para ver o histórico de movimentação de uma tag específica.
 - Relatório simples de tempo médio de permanência por câmara.
-

Requisitos e Escopo

Esta seção detalha os requisitos funcionais e não funcionais que definem o escopo do projeto.

Requisitos Funcionais (RF)

- **RF-01: Captura de Dados RFID**
 - O sistema deve receber e interpretar os sinais das antenas RFID.
 - Deve registrar o ID único de cada tag lida.
 - Deve identificar qual antena realizou a leitura.
- **RF-02: Registro de Movimentação**
 - Ao entrar em uma câmara, o sistema deve registrar o ID da tag rfid, ID da câmara, ID da antena, frequência, data/hora de entrada e o supervisor responsável no turno.
 - Ao sair da câmara, o sistema deve registrar o horário de saída na mesma entrada do banco de dados.
- **RF-03: Dashboard de Status em Tempo Real (Gêmeo digital)**
 - O frontend deve exibir uma lista de câmaras.
 - Para cada câmara, o sistema deve listar todas as tags RFID que estão atualmente dentro dela.
 - A visualização deve ser atualizada automaticamente ou com mínima intervenção do usuário.
- **RF-04: Consulta de Histórico**
 - Permitir que o usuário pesquise pelo ID de uma tag.
 - Exibir o histórico completo de movimentações da tag pesquisada, incluindo todas as câmaras pelas quais passou, com horários de entrada e saída.
- **RF-05: Relatórios Básicos**
 - O sistema deve ser capaz de calcular e exibir o tempo médio de permanência das tags em cada uma das câmaras.

Requisitos Não Funcionais (RN)

- aspectos de instalação do kit para cada câmara; inicialização do sistema;
-
- **RN-01: Desempenho**
 - O sistema deve processar uma leitura de tag e registrá-la no banco de dados em menos de 2 segundos.
 - O dashboard deve carregar os dados de status em menos de 5 segundos.
- **RN-02: Confiabilidade e Disponibilidade**
 - O sistema deve operar 24/7.
 - O sistema deve possuir mecanismos de log para rastrear erros no backend.
- **RN-03: Usabilidade**
 - A interface do frontend deve ser intuitiva, permitindo que um supervisor aprenda a operar as funções básicas.
 - O dashboard deve ser responsivo, adaptando-se a telas de computadores e tablets.
- **RN-04: Segurança**
 - O acesso ao front end deve ser protegido por autenticação (login/senha).

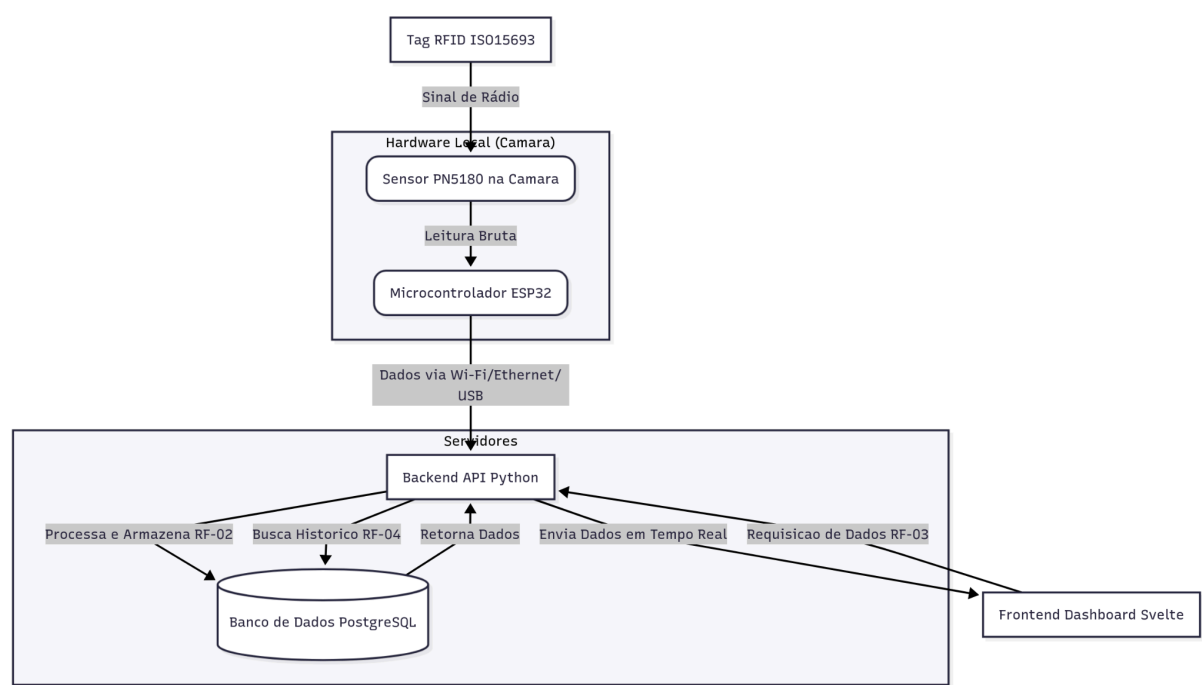
- Diferentes níveis de permissão (ex: operador, supervisor, gerente) poderão ser implementados em fases futuras.

Campos do Banco de Dados

Campo	Descrição
EPC (Electronic Product Code)	Identificador único da tag (ex: E2801170000002094A5B21B7)
Timestamp	Data/hora exata da leitura
Reader ID	Qual leitor RFID processa o sinal
RSSI (Received Signal Strength Indicator)	Intensidade do sinal — ajuda a estimar a distância da tag
Frequência (MHz)	Frequência exata usada na leitura
Phase Angle (graus)	Pode ser usada para cálculos mais avançados de localização
Local / Zona Lógica	Ex: “Câmara Fria A”, “Caminhão 02”, “Portão de Saída”
Operador / Funcionário Supervisor	Identificado via sistema ou login no backend
Cliente / Proprietário	Identificador único do cliente ou proprietário do produto
Produto	Nome do produto que foi lido

Fluxos de Processo

Arquitetura Geral do Sistema



Código do diagrama Mermaid

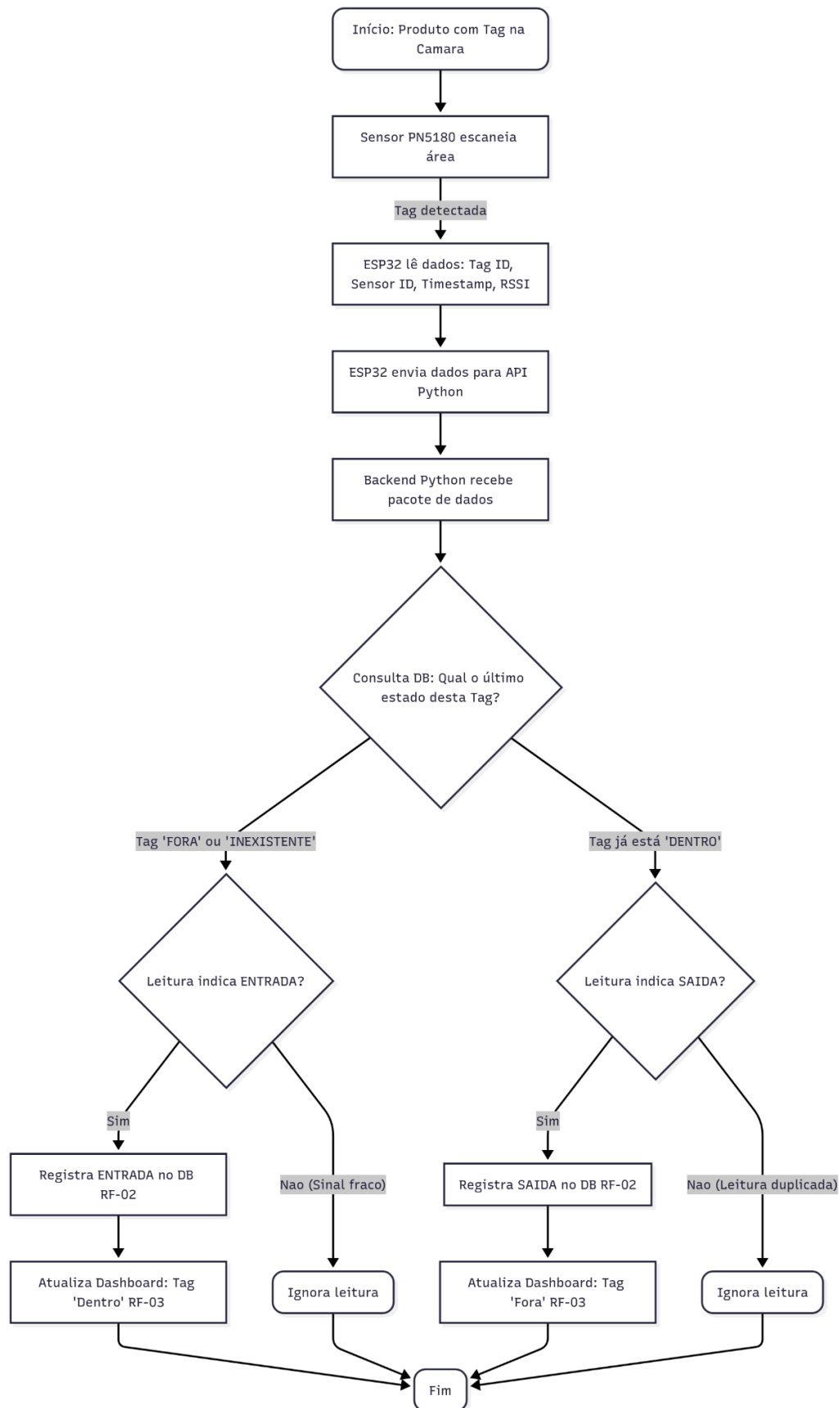
```
graph TD
    A[Tag RFID ISO15693] -- "Sinal de Rádio" --> B(Sensor PN5180 na Camara);
    B -- "Leitura Bruta" --> C(Microcontrolador ESP32);
    C -- "Dados via Wi-Fi/Ethernet/USB" --> D[Backend API Python];
    D -- "Processa e Armazena RF-02" --> E[(Banco de Dados PostgreSQL)];
    F[Frontend Dashboard Svelte] -- "Requisicao de Dados RF-03" --> D;
    D -- "Busca Historico RF-04" --> E;
    E -- "Retorna Dados" --> D;
    D -- "Envia Dados em Tempo Real" --> F;

    subgraph "Hardware Local (Camara)"
        B
        C
    end
```

```
    B
    C
end
```

```
subgraph "Servidores"
    D
    E
end
```

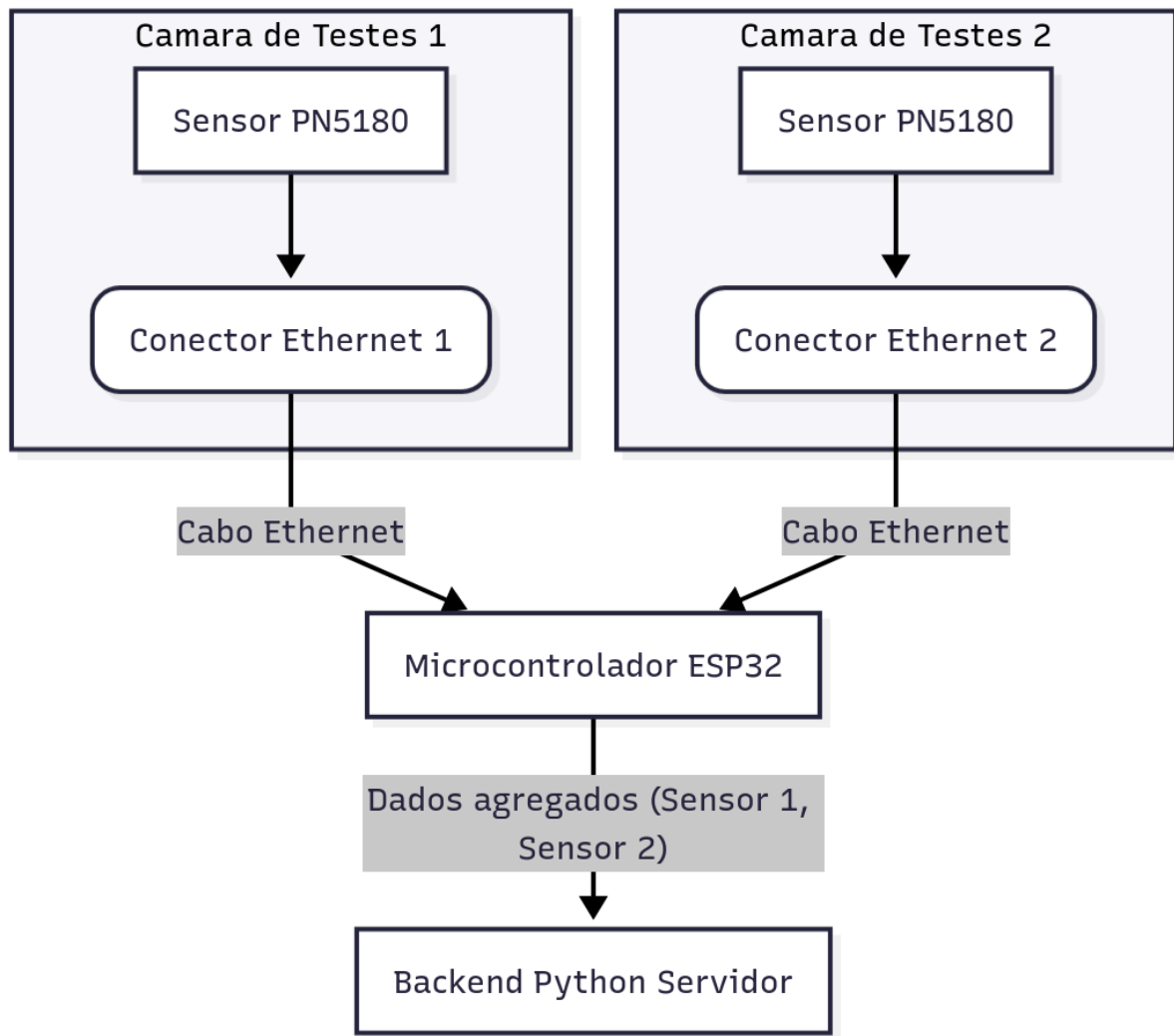
Fluxo Lógico de Processamento



Código do diagrama mermaid:

```
None
flowchart TD
    Start([Início: Produto com Tag na Camara]) --> Scan[Scan[Sensor PN5180 escaneia área]];
    Scan -- "Tag detectada" --> Read[Read[ESP32 lê dados: Tag ID, Sensor ID, Timestamp, RSSI]];
    Read --> Send[Send[ESP32 envia dados para API Python]];
    Send --> Receive[Receive[Backend Python recebe pacote de dados]];
    Receive --> CheckDB[CheckDB{Consulta DB: Qual o último estado desta Tag?}];
    CheckDB -- "Tag 'FORA' ou 'INEXISTENTE'" --> IsEntry[IsEntry{Leitura indica ENTRADA?}];
    IsEntry -- "Sim" --> RegisterEntry[RegisterEntry[Registra ENTRADA no DB RF-02]];
    RegisterEntry --> UpdateFE1[UpdateFE1[Atualiza Dashboard: Tag 'Dentro' RF-03]];
    UpdateFE1 --> Stop1([Fim]);
    IsEntry -- "Nao (Sinal fraco)" --> Ignore1[Ignore1[Ignora leitura]];
    Ignore1 --> Stop1;
    CheckDB -- "Tag já está 'DENTRO'" --> IsExit[IsExit{Leitura indica SAIDA?}];
    IsExit -- "Sim" --> RegisterExit[RegisterExit[Registra SAIDA no DB RF-02]];
    RegisterExit --> UpdateFE2[UpdateFE2[Atualiza Dashboard: Tag 'Fora' RF-03]];
    UpdateFE2 --> Stop2([Stop]);
    IsExit -- "Nao (Leitura duplicada)" --> Ignore2[Ignore2[Ignora leitura]];
    Ignore2 --> Stop2;
```


Fluxo de Múltiplos Sensores



Código do diagrama Mermaid:

None

```
graph TD
```

```
    subgraph "Camara de Testes 1"
```

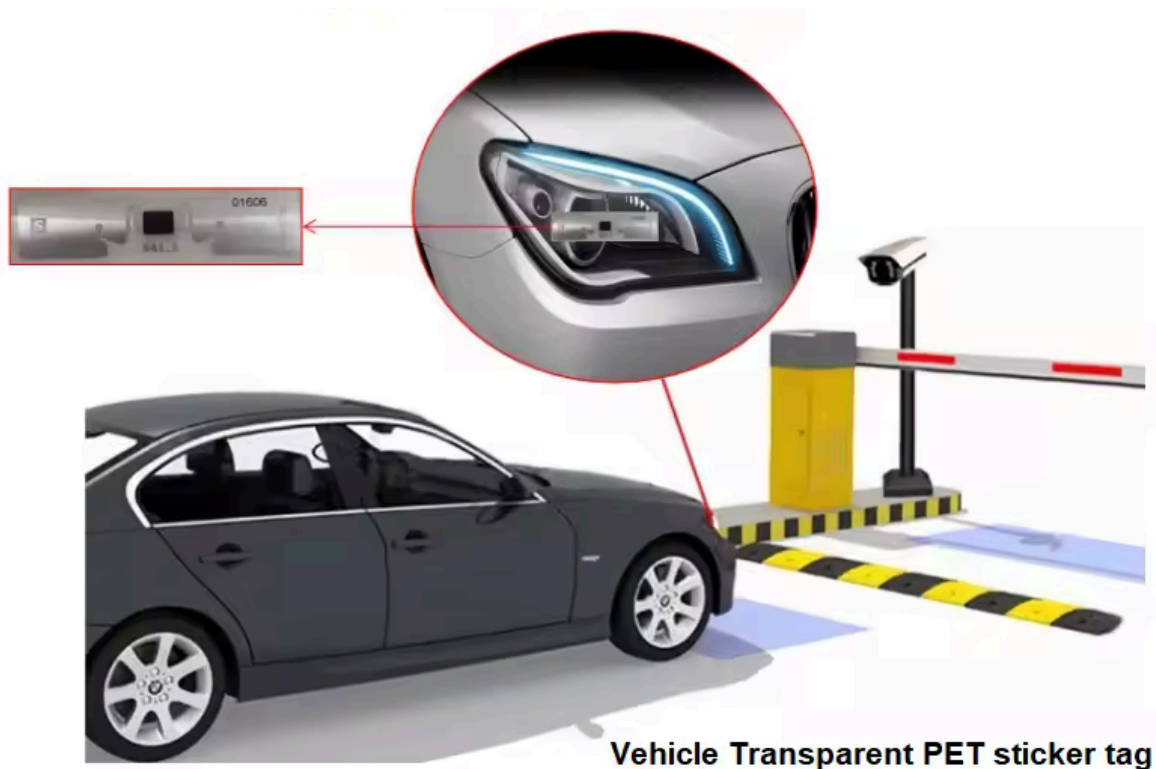
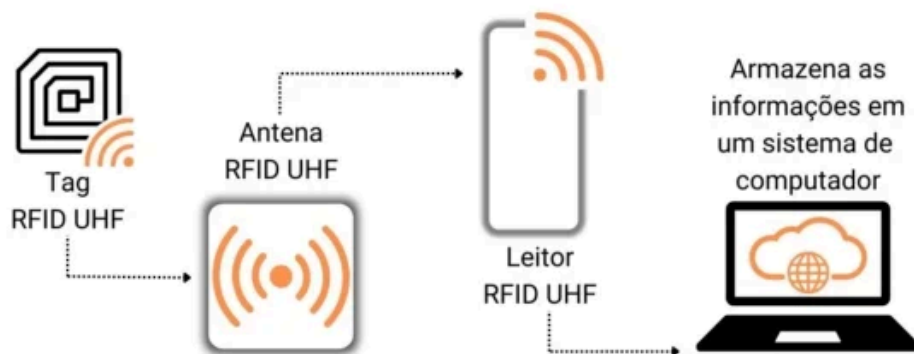
```
        S1[Sensor PN5180] --> E1(Conector Ethernet 1);
```

```
    end
```

```
    subgraph "Camara de Testes 2"
```

```
S2[Sensor PN5180] --> E2(Conector Ethernet 2);  
  
end  
  
E1 -- "Cabo Ethernet" --> ESP32[Microcontrolador ESP32];  
  
E2 -- "Cabo Ethernet" --> ESP32;  
  
ESP32 -- "Datos agregados (Sensor 1, Sensor 2)" --> Backend[Backend  
Python Servidor];
```

Hardware utilizado



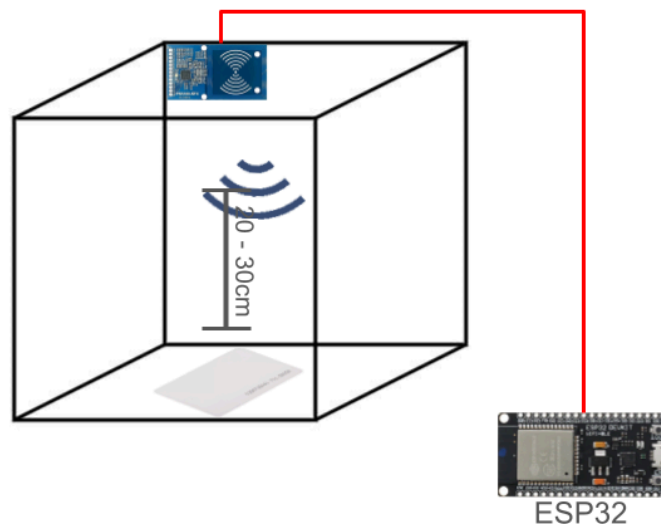
Para a execução deste projeto em seu estado de testes inicial foi escolhido os seguintes equipamentos:

- Módulo sensor ISO15693 PN5180: Por aderir ao protocolo ISO15693 tem maior alcance de leitura, podendo ser usado a diferentes distâncias dentro de um raio de 20cm (A depender da taxa de interferência entre os materiais)

- Módulo microcontrolador ESP32: O ESP32 possui ótima compatibilidade com python, além de vir com módulos wifi e bluetooth, permitindo maior variedade de conectividade para testes.
- Conector de cabo Ethernet: Para realizar conexões entre sensores e microcontroladores com maior distância entre si, poderá ser utilizado um conector de cabo Ethernet para transferir os sinais para o microcontrolador.

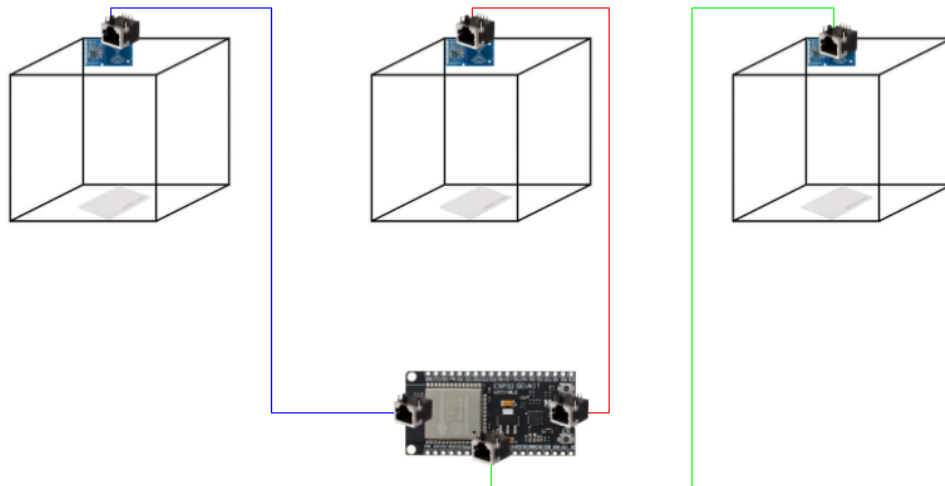
Configuração inicial prevista (Módulo único)

O módulo de sensor será acoplado a uma câmara de testes. Será realizada a conexão entre os módulos de sensor e o microcontrolador de modo a processar a entrada e saída de TAGS RFID no alcance do sensor para processamento em python e o registro no banco de dados.



Configuração prevista para mais de um módulo

Neste caso, poderá ser utilizado os conectores de cabo Ethernet para a integração entre os módulos a um único microcontrolador.



Cronograma estimado

Mês	Fase Principal	Atividades Chave & Entregas
Mês 1-4	Planejamento, Análise	- Detalhamento final dos requisitos com os stakeholders. Definição da arquitetura do sistema (Backend, DB, Frontend). Seleção e compra do hardware (antenas, leitores, tags).
Mês 4-6	Desenvolvimento do Core (Backend & Banco de Dados)	- Modelagem e implementação do banco de dados PostgreSQL. - Desenvolvimento das APIs (Python) para: - Receber dados das antenas. - Registrar entradas e saídas. - Fornecer dados para o frontend. - Implementação da lógica de negócio para identificar entrada/saída.
Mês 6-7	Desenvolvimento do Frontend e Integração	- Desenvolvimento dos wireframes e layout do dashboard. - Criação dos componentes Svelte (lista de câmaras, visualização de tags, tela de busca). - Integração do frontend com as APIs do backend. - Testes completos de ponta a ponta (leitura da tag até a exibição na tela).
Mês 8	Testes, Refinamento e Implantação Piloto	- Instalação física das antenas em uma área piloto (ex: 2 ou 3 câmaras). - Realização de testes de sistema em ambiente real. - Coleta de feedback

		dos primeiros usuários (supervisores piloto). - Correção de bugs e ajustes de usabilidade.
Mês 9	Implantação Final, Treinamento e Encerramento	- Implantação do hardware em todas as câmaras planejadas. - Lançamento oficial do sistema para toda a produção. - Treinamento formal de todos os supervisores e operadores envolvidos. - Finalização da documentação técnica e dos manuais do usuário.